Design Document

For

Fill My Fridge

By: Ana Knutson, Billy York, David Litton, Yuliya Kazadoi

Version 1.0 draft I

December 12th, 2024

Table of Contents

Revision History 3
Quality Attributes and their Scenarios 3
1. Safety 3
Tactics: 3
Scenarios: 4
2. Usability 4
Tactics 4
Scenarios: 5
3. Security 6
Tactics 6
Scenarios: 7
4. Integrity 7
Tactics 8
Scenarios: 8
Initial Implementation Plan 10
Overview 10
Architecture 10
Pros and Cons of our Tactics 11
Modules, Tactics, and their Interfaces 15
Views 26
Component Diagram 26
Data Dictionary 27
Data Flow Diagram 28
Entity Relationship Diagram 29
Sequence Diagram 29
Site Map 30
UML/Class Diagram 31
Use Case Diagram – Alternate Routes to Adding/Importing Recipes 32

Work Context Diagram 33
Add Ingredients to Recipe Mocks 34
Glossary 35

Revision History

Contributors	Date	Reason for Change	Version
Ana, Billy, David, Yuliya	11/18/2024	Initial draft for design document	1.0 Draft I
David	12/12/2024	Moved content from implementation plan to design plan	1.0 Draft II

Quality Attributes and their Scenarios

1. Safety

Safety is our top priority because, as an app that helps people maintain their health and plan meals, we want to make sure no health issues are caused by user's unnoticed mistake. That is why we want to make sure if the user adds a recipe with ingredient(s) that they are allergic to, the app will notify them about that while also allowing them to leave it (in case they are cooking it for someone else).

Tactics:

Unsafe State Detection:

- Sanity Check: Ensures that all added ingredients are verified against the user's allergen list to prevent accidental inclusion of allergens. This proactive validation minimizes the risk of harmful or unintended ingredient additions during recipe creation or import.

Recovery:

- Rollback: Allows the user to reverse actions, such as canceling the addition of an ingredient or the import of a recipe. This feature preserves the app's usability by enabling users to undo changes and continue seamlessly without committing to potentially problematic modifications.

Scenarios:

Source of Stimulus	User
Stimulus	The user adds an ingredient they are allergic to.
Environment	Normal operation
Artifact	Ingredient database
Response	Recognize the unsafe state, notify user by displaying the warning with options to cancel adding ingredient or continue normal flow, log the unsafe state
Response Measure	The response is performed within 100ms, and the user cannot go on until they choose one of two options on the warning

Source of Stimulus	User
Stimulus	The user imports a recipe containing an ingredient they are allergic to.
Environment	Normal operation
Artifact	Ingredient database
Response	Recognize the unsafe state, notify user by displaying the warning with options to cancel recipe import or continue normal flow, log the unsafe state
Response Measure	The response is performed within 100ms, and the user cannot go on until they choose one of two options on the warning

2. Usability

Usability is our second priority because when the user is adding a recipe to our application, **FMF**, we want the process to be a simple and understandable one that will allow the user to be able to add a recipe in an intuitive way. This will allow for **FMF** to be more likeable by users if the interfaces are clean and work how most people would assume them to. This will ensure that the users have the best experience that we can provide to ensure they keep going back and continue to use our application.

Tactics

FMF shall benefit from the use of a pause/resume. This will allow for the system to pause or resume to ensure that resources can be reallocated without losing progress that has been made by the user in regards to filling out the form to create a new recipe. This will

be used in case the user gets sidetracked and exits the application before they have finished creating their new recipe, or if they simply need to check something else on their smartphone before finishing adding a new recipe.

The program also shall use a maintain task model. This model is beneficial for our system, as it will be used to try and predict what the user is typing when adding new ingredients. This will ensure that the program is even simpler for the user as it gives them the ability to simply type in a few characters and have it provided three of the closest possibilities that our system has found using the **USDA Foundational Foods and Branded Foods Databases**. Especially for recipes with many ingredients, this will allow for the user to create a recipe even quicker.

Even more, **FMF** will use the cancel tactic. This means that the user can cancel or stop an action that they may have been in the middle of. This allows the user to have control over if they changed their mind and want to end a process, also allowing the system to free up resources that may have been allocated for the process the user was in. This means that if the user in in the middle of adding a new recipe to the application, while they are adding the recipe, if the user ends up changing their mind and deciding not to add the recipe anymore, they can simply cancel.

Scenarios:

Source of Stimulus	User
Stimulus	The user indicates they want to cancel adding the recipe
Environment	Normal Operation
Artifact	**Recipe Database**
Response	The system deallocates resources used such as data the user added to the recipe, and the information is discarded.
Response Measure	The user is satisfied 85% of the time by all of the recipe being deleted on a cancel, and the 15% dissatisfaction is related to factors such as an accidental cancellation.

Source of Stimulus	User
Stimulus	The user types in an ingredient they want to add
Environment	Normal Operation
Artifact	**USDA Branded and Foundational Foods Databases**

Response	The system gives the user the top 3 food items that contain the words that the user has currently entered
Response Measure	The user learns that they can enter only the first few letters of an ingredient within the first 3 times they attempt to add an ingredient.

Source of Stimulus	User
Stimulus	The user closes the application while they are still in the process of adding a recipe.
Environment	Normal Operation
Artifact	**Recipe Database**
Response	The system saves the current draft of the recipe that the user was working on
Response Measure	The data that is saved is data that had been entered 1 second before the user exited the application

3. Security

Security is our third most important quality attribute because for the ability to connect to grocery stores, we will need to save authentication data, such as an OAuth bearer token. We do trust the user enough to not be malicious to their own account data and our local first philosophy provides a higher level of security than the average always connected mobile app, however there is a single attack vector identified through a connection to the wider internet, importing recipes. Other potential vectors would be malware on the device, or man-in-the-middle attacks. We do not believe our user's data would be directly useful to any malicious actor, however we believe it would be a tool they could use to attack a larger chain and perhaps gain credit card information.

Tactics

Our most important tactic will be to Resist Attacks by Encrypting Data at rest, Authenticating Actors, Validating Input, and Limiting Access. The Grocery Store Authentication data is only used in a specific scenario, so there is no need to have it constantly ready at a moment's notice. By encrypting the data, we are hardening the most valuable data our system contains.

Each time the Authentication Data is requested from the database, the system will use the mobile device's built in security systems to perform authentication of the user, such as with device biometrics. Only upon device authentication will the Grocery Store Authentication Data be decrypted.

We will also Validate Input. While we trust the user to input valid data, we will still validate it as there is no reason not to since we need to validate input gathered from the internet. We anticipate the possibility of a cross-site scripting attack, possibly hidden in the source code of a website so the user wouldn't even see it, that could perform an SQL injection on our device hosted database. As a result, we will need to validate every body returned in a HTTP response.

Lastly, we will Limit Access by using as little SQL as needed by the device's APIs, opting instead to use DAO's and ORM's provided by the API's. These DAO and ORM usages will need to be compiled into the app and therefore make this any attacks coming through this vector be significantly more sophisticated in their design.

Scenarios:

Source of Stimulus	User (attempting to import a recipe)
Stimulus	Capture Grocery Store Authentication Data
Environment	Receiving HTTP Response with malicious XSS body
Artifact	Data regarding User Grocery Store Authentication
Response	User Grocery Store Authentication data is inaccessible without device level authorization.
Response Measure	Each Recipe Import is logged. Each attempted access of Grocery Store Authentication is logged. If there are two events within less than 2 seconds of each other the system notifies the User of the attack.

4. Integrity

FMF depends on internal data to be in a understandable format. Whether it is the user supplied recipes, the recipes scraped from the web, or the nutritional information provided by the **USDA database**. Additional data integrity importance comes from our many systems consistently saving and reading data, like the pantry, grocery list and meal plan. Almost all of our systems depend on reading correct data and then must save the

data back in a correct state. Lastly **FMF** will be deployed on multiple operating systems and using the running operating system to determine where to store files on the **device**. It is important that the program can find any file it needs regardless of the installed operating system.

Tactics

Limited Dependencies:

- Abstracting Common Services: develop a system to handle saving and retrieving files from storage regardless of the operating system's **file system**. For example, a file name can be sent to this system and the file will be returned or a **relative location** and file contents are pushed to the system and it handles where to place the file. This tactic is effective for this problem as it will reduce the code written to write and read files and will ensure even if there is an issue the issue is consistent for all files read and wrote.
- Adhere to Standards: scheme.org has developed **xml** schemes for different use cases. Sticking to their standards should make scrapped data mech nicer with our internal state.

Scenarios:

Source of Stimulus	User
Stimulus	The user is saving their recipe or opening a draft of a previously made recipe.
Environment	Runtime
Artifact	Recipe file/xml
Response	The operating independent file saving system will determine where to safe the file in the case or a save. In the case of a read/load the system will locate the file on the operating system and return the location.
Response Measure	The file saving system being self-contained will cut down on individual systems needing to locate files and will only need to process parsing input or reading a file from a path. This will cut down on duplicate code by 50%

Source of Stimulus	User
Stimulus	The user saves the recipe form.

Environment	Runtime
Artifact	Recipe file/xml
Response	The resulting file/record of the recipe form will be stored in an xml format corresponding to the scheme.org recipe.
Response Measure	Storing recipes as a standard xml format will make parsing web scraped data 75% easier given the website being scrapped follows the same standard.

Initial Implementation Plan

Overview

Based on the Systems Requirement Specification (SRS) and the Design Document, we will be implementing the recipe creation use case (UC-2) and some surrounding features. The main modules we identified align with many of the steps a user will need to take when creating a recipe: adding ingredients, tags, cooking methods, time, servings, etc. Besides that, we will also have modules that align with actions that enhance the recipe collection feature, such as importing recipes from a recipe website, adding a new ingredient, and creating nutritional information for that new ingredient, along with storing and displaying the user's collected recipes. We will be prioritizing Safety, Usability, Security, and Integrity by implementing tactics that enforce these qualities. Examples of some of the tactics we will be implementing include sanity check, encryption, input validation, user authentication and more. Safety will be our main priority to protect users who have dietary restrictions or food allergies. A key aspect of our system will be warning users with these specified restrictions whenever they try to add a recipe that contains an ingredient to which they are allergic. This will be implemented in "Add ingredient" and "Import recipe" modules. The application will be built using Model-View-Controller (MVC) design pattern for separating main app logic from interface elements and data management. This will make it easier to maintain, test, and add new modules to the application in the future and support future development.

Architecture

In our application, Fill My Fridge, we will be using event-driven architecture for the front end to allow for a responsive interface. This pattern is suitable for handling the many possible user actions that will be implemented such as entering information, selecting options, and pressing buttons. With this approach, the UI can respond to user actions immediately by triggering necessary updates or respond with requested information. For the backend, we have chosen a data-centric architecture, as our app is centered around information that the user enters that our application will then use to provide the user with an experience that feels personalized to this information. Since our app stores a large amount of data (such as recipes, food items from USDA databases, and more), we need this approach to

ensure there is integrity and consistency within the data. This architecture allows different modules to access and update data efficiently, which is very important for making sure data is accurate and up to date across the application. Some of the tradeoffs for these patterns include the possible performance overhead in situations when the system cannot handle the user events, or scalability limitations as the app stores more and more data which may lead to operations taking longer to complete, affecting overall application performance.

Pros and Cons of our Tactics

Sanity Check

- Pros:
  - Increased safety. Helps reduce the chance of changes that are unwanted.
  - User doesn't need to spend time checking recipes manually, the app makes it easier to track allergens without putting extra effort.
- Cons:
  - Lower performance. Checking every entered ingredient might slow down the app.
  - Needs additional logic implementation for matching allergens to the ingredients.

Rollback

- Pros:
  - Better user experience. Makes it easier for users to undo the actions without having to perform extra steps and change the system state.
  - Easier error correction. The user can undo the action they did not want to perform.
- Cons:
  - Implementation complexity. The system shall keep track of previous states and be able to switch between them.
  - Higher resource demand. The system needs to store previous states to be able to perform rollback, which requires additional storage.

Pause/Resume

- Pros:
  - Users feel like they have more control over the software because they can interrupt long running processes and can pause them to later come back to them.
  - Allows for more flexibility since the user can prioritize other tasks by pausing processes and controlling the order of which processes run.
- Cons:
  - Some tasks may not be able to successfully be paused and allowing the user to have the option of pausing them may cause issues in the system.
  - If a user is frequently pausing or unpausing processes, it may become confusing as to what processes have finished, what the status of them is, and it could cause an incorrect order of processes to run resulting in incorrect states or calculations.

Maintain Task Model

- Pros:
  - More efficient interactions between the user and the system since the system is attempting to guess what the user is inputting and can save the user time.
  - Help align the user's expectations of the system and the system itself since the system is consistent and is suggesting things based upon what the system is expecting the user to do or enter.
- Cons:
  - Can increase complexity since it may be complex to implement models that are predictive based on user input.
  - Can cause performance issues since continuously updating and querying based upon the users input uses additional resources.
  - May be unnecessary and could annoy the user as sometimes filling things in preemptively or giving the user incorrect suggestions can lead to user dissatisfaction.

Cancel

- Pros:
  - Gives the user more control as they can cancel a task if they want to

    - Helps prevent unintended actions since the user can cancel an action before it is fully executed.
    - Prevents the user from getting frustrated. If the user was unable to cancel, they might get annoyed with the system since, when they make an error, there may not be a straightforward way to stop the error.
- Cons:
    - Canceling an action could result in data loss. This could lead to the user having to redo many previous actions to correct the cancel operation and resubmit substantial portions of data.
    - The user may not understand what the ramifications of a cancel require, thus causing the user to cancel when they did not want as much progress to be lost, or for the state of their program to return to as early of a state as it does.

Encrypt Data

- Pros:
    - Keeps data secure in the event of a breach.
    - Enables more trust from the user.
- Cons:
    - Encrypting and decrypting data will take the form of performance hit.
    - If Encryption keys are ever lost or become degraded, then the data is unrecoverable.

Authenticate Actor

- Pros:
    - Adds a level of confidence to know if the user is who they say they are.
    - Acts as a form insurance in the event the user's authentication data is compromised (CYA)
- Cons:
    - Only guarantees the user has the correct authentication data.
    - May rely on external authentication systems to work properly or involves creating a sophisticated module that is not integral to the app's functionality.

Validate Input

- Pros:
    - Prevents specific attack vectors.

    - Helps with Integrity to keep data uniform.
- Cons:
    - Impossible to prevent every kind of script injection.
    - Will cause a performance hit as the function needs to take an extra, usually unnecessary step.

Limit Access

- Pros:
    - Creates a single point where valuable data is accessed so it can be secured further.
    - Prevents unnecessary components that may be compromised from being able to access the data.
- Cons:
    - Increases coupling on any component that needs to access the valuable data.
    - Does not fully guarantee protection of the data, just limits its exposure.

Abstracting Common Services

- Pros:
    - Reduced duplicate code. This can lead to more consistent outcomes even regardless of the correctness of the code as more systems will rely on a single implementation.
    - Changes made to the abstraction will affect all implementations thus reducing the need to change multiple systems.
- Cons:
    - Cost for over abstraction. Not every system needs an abstraction. Some systems may be specialized and abstracting them out serves no purpose to other systems and leads to a more nuanced system and wasted time.
    - Debugging becomes more difficult as the abstractions increase the number of unique places a system can fail.
    - Over generalization of an abstraction can lead to limited functionality of the system.

Adhere to Standards

- Pros:

    - Future proofing. As More systems and products use similar standards the interoperability increases as a side effect.
    - As a system conforms to a standard interacting with that system becomes more predictable.
    - Consistency. Systems will not have proprietary documentation for standards.
- Cons:
    - Less flexibility. Conforming to standards might require you to implement the system a particular way. Rather than having a perfect system designed for your needs you might have to adapt to the standard to implement it.
    - Though conforming to a standard increases the future proofing of the product/system that depends on the standard sticking around.
    - Can be costly to adhere old systems to a new standard.

Modules, Tactics, and their Interfaces

Add New Recipe Button Action:

- Tactic(s):
    - Abstracting common services: Adding a system for creating buttons where it pertains to the button object or just styling will make sure all buttons look the same.
- Interface(s):
    - createButton(String label, Function action, Object styleOptions): Button
    - createNewRecipe(Object template): Recipe

Determine if Recipe Name is Unique:

- Tactic(s):
    - Validate Input: Ensures that recipe names follow rules (like being unique) before they are saved, maintaining data integrity.
    - Sanity Check: Helps prevent duplicate recipe entries by checking for uniqueness automatically.
    - Adhere to Standards: Enforces naming standards, ensuring uniformity, and preventing unexpected behavior from special characters or duplicates.
- Interface(s):
    - isUnique(String): Boolean

Add Ingredient Action:

- Tactic(s):
    - Rollback: being able to cancel adding the ingredient if the user decides to
    - Sanity Checking: checking if the ingredient contains allergens.
    - Cancel: the user may determine that they may not want to add a new ingredient, so cancelling will allow for the user to cancel this action.
    - Maintain User Task Model: giving users suggestions or guessing what the user is entering can allow the user to add an ingredient even quicker.
- Interface(s):
    - addIngredient(Object): Boolean

Select Unit Action:

- Tactic(s):
    - Maintain User Task Model: Suggests commonly used units or predicts unit selection based on the ingredient type, making the process faster and more intuitive.
    - Adhere to Standards: Ensures that units follow consistent formatting (e.g., metric vs. imperial) and are universally recognized.
- Interface(s):
    - getSelectedUnit(): String

Unit Table/Enum:

- Tactic(s):
    - Abstracting common services: Removing the definition of units from the code will make maintaining the specified units easier.
- Interface(s):
    - getUnitList(): Array
    - getUnit(String): Object

Add Quantity Action:

- Tactic(s):
    - Sanity Checking: making sure the quantity does not go below and above the limits.
    - Cancel: the user may determine that they may not want to add the quantity, so cancelling will allow for the user to cancel this action.

- Interface(s):
    - getQuantity(): Number

Search Ingredients Action:

- Tactic(s):
    - Abstracting common services: Searches are used throughout the app. It may be useful to abstract the core functionality of a search out.
    - Maintain User Task Model: giving users suggestions or guessing what the user is entering can allow the user to search for an ingredient even quicker.
- Interface(s):
    - updateResults(String): Array
    - selectIngredient(Object): Boolean

Create Ingredient Action:

- Tactic(s):
    - Sanity Checking: Ensures that the new ingredient is valid before it is created (e.g., checking for allergens, proper format).
    - Rollback: Allows the user to undo the creation of the ingredient if they change their mind.
    - Validate Input: Ensures that the input data for the ingredient meets required standards (e.g., no invalid characters).
- Interface(s):
    - createIngredient(Object): Boolean

Create Nutrition Info:

- Tactic(s):
    - Abstracting common services: Adding a system for creating buttons where it pertains to the button object or just styling will make sure all buttons look the same.
    - Cancel: the user may determine that they may not want to create the new ingredient, so cancelling will allow for the user to cancel this action.
- Interface(s):
    - createNutritionInfo(Object): Boolean

Nutrition Database:

- Tactic(s):
    - Abstracting Common Services: Centralizes access to the nutrition data, making it easier to maintain and retrieve nutrition information across the app.
    - Adhere to Standards: Ensures consistency in the format and structure of nutritional data, following any relevant dietary or regulatory guidelines.
- Interface(s):
    - createNutritionInfo(Object): Boolean
    - getNutritionData(String): Object

Ingredient Database:

- Tactic(s):
    - Adhere to standards: ensuring that the database follows good standards. And that we are not reinventing the wheel when it comes to best practice.
- Interface(s):
    - findNutrition(String): Object
    - insertNutrition(Object): Boolean

Add Instruction Action:

- Tactic(s):
    - Adhere to standards: ensuring that the database follows good standards. And that we are not reinventing the wheel when it comes to best practice.
- Interface(s):
    - findIngredient(String): Object
    - insertIngredient(Object): Boolean

Add Recipe Tag Action:

- Tactic(s):
    - Abstracting common services: Adding a system for creating buttons where it pertains to the button object or just styling will make sure all buttons look the same.
    - Cancel: the user may determine that they may not want to add a new instruction, so cancelling will allow for the user to cancel this action.
- Interface(s):

- newInstruction(Object): Boolean

Search Recipe Tags Action:

- Tactic(s):
  - Abstracting Common Services: Centralizes the search functionality for recipe tags to ensure consistent behavior across the app.
  - Maintain User Task Model: Provides suggestions as the user types, improving search efficiency for recipe tags.
- Interface(s):
  - updateTagResults(String): Array
  - selectRecipeTag(Object): Boolean

Create New Tag Action:

- Tactic(s):
  - Abstracting common services: Searches are used throughout the app. It may be useful to abstract the core functionality of a search out.
  - Maintain User Task Model: giving users suggestions or guessing what the user is entering can allow the user to search for a tag even quicker.
- Interface(s):
  - updateResults(String): Array
  - selectTag(Object): Boolean

Recipe Tag Table:

- Tactic(s):
  - Cancel: the user may determine that they may not want to create the new tag and want to use a previously made one, so cancelling will allow for the user to cancel this action.
- Interface(s):
  - createTag(Object): Boolean

Select Cooking Method Action:

- Tactic(s):
  - Cancel: the user may determine that they may not want to select a cooking method, so this allows the user to stop this action.

    - Maintain User Task Model: giving users suggestions or guessing what the user is entering can allow the user to search for a cooking method even quicker.
- Interface(s):
    - getCookMethod(): String

Cooking Method Enum:

- Tactic(s):
    - Abstracting common services: Removing the definition of units from the code will make maintaining the specified units easier.
- Interface(s):
    - getCookingMethods(): Array
    - getCookingMethod(String): Object

Add Cooking Time Action:

- Tactic(s):
    - Cancel: the user may determine that they may not want to add a cooking time, so this allows the user to stop this action.
- Interface(s):
    - cookTime(Number): Boolean

Add Number of Servings Action:

- Tactic(s):
    - Cancel: the user may determine that they may not want to add a number of servings, so this allows the user to stop this action.
- Interface(s):
    - numServings(Number): Boolean

Allergen Identifier:

- Tactic(s):
    - Cancel: the user may determine that they may not want to worry about the allergen that is in their dietary restrictions as it could be for a friend, and they will not be eating it. This would allow the user to cancel the identification of the allergen and continue creating the recipe.

- Interface(s):
    - allergenWarning(Object): Boolean

Allergen Database:

- Tactic(s):
    - Adhere to standards: ensuring that the database follows good standards. And that we are not reinventing the wheel when it comes to best practice.
- Interface(s):
    - findAllergen(String): Object
    - insertAllergen(Object): Boolean

Recipe Save Action:

- Tactic(s):
    - Abstracting common services: Adding a system for creating buttons where it pertains to the button object or just styling will make sure all buttons look the same.
- Interface(s):
    - saveRecipe(Object): Boolean

Recipe Verification Action:

- Tactic(s):
    - Validate Input: before the recipe is saved, the contents of it are validated and sanitized to prevent any malicious code injections as well as verify the data follows the format for the database.
- Interface(s):
    - verifyRecipe(Object): Boolean

Recipe Cancel Action:

- Tactic(s):
    - Cancel: the user may determine that they may not want to create the recipe, so cancelling will allow for the user to delete and stop creating the new recipe.
- Interface(s):

- cancelRecipe(): Boolean

Recipe Save Draft Action:

- Tactic(s):
  - Rollback: Allows the user to save a draft and come back to it later, providing a way to undo or modify changes.
  - Cancel: Lets the user decide to stop saving the draft if they change their mind.
- Interface(s):
  - saveDraft(Object): Boolean

Recipe Render Action:

- Tactic(s):
  - Abstracting Common Services: Ensures consistent rendering of recipe details across the app by centralizing the rendering logic.
  - Maintain Task Model: Helps in presenting recipe information dynamically, adjusting the view based on the state of the recipe data.
- Interface(s):
  - renderRecipe(Object): Boolean

Import Recipe Action:

- Tactic(s):
  - Abstracting common services: Adding a system for creating buttons where it pertains to the button object or just styling will make sure all buttons look the same.
  - Pause/Resume: if the importing is taking too long, the user may want to pause the process and do something else before resuming the import of the recipe.
  - Cancel: the user may determine that they may not want to import the recipe in the middle of it importing, so cancelling allows the user to perform other actions.
- Interface(s):
  - importRecipe(Object): Boolean
  - pauseImport(): Boolean
  - resumeImport(): Boolean

- cancelImport(): Boolean

Imported Recipe Validator:

- Tactic(s):
  - Sanity Checking: making sure the recipe website follows basic standards.
- Interface(s):
  - validateImportedRecipe(Object): Boolean

Web Scraping Module:

- Tactic(s):
  - Abstracting Common Services: Ensures that web scraping logic is centralized and reusable across the app, making it easier to manage and maintain.
  - Sanity Checking: Validates the data scraped from websites to ensure it meets the expected format and standards before processing it further.
- Interface(s):
  - scrapeWebPage(String): Object
  - parseRecipeData(Object): Object

URL Validator:

- Tactic(s):
  - Validate Input: At least for 1.0 and any immediate follow-up releases, the plan is to only allow specific websites that are trusted by the developers and follows consistent formatting to make scraping easier. Because of this, the URLs will be validated that they are directed to one of these trusted websites.
- Interface(s):
  - validateURL(String): Boolean

Recipe Database:

- Tactic(s):
  - Adhere to standards: ensuring that the database follows good standards and that we are not reinventing the wheel when it comes to best practice.
- Interface(s):
  - findRecipe(String): Object
  - insertRecipe(Object): Boolean

- updateRecipe(Object): Boolean
- deleteRecipe(String): Boolean

Ingredient List Table:

- Tactic(s):
  - Adhere to standards: Follow internal standards for database table norms will help ensure ACID or any other database consistency metrics we deem important.
- Interface(s):
  - findIngredientList(String): Array
  - insertIngredientList(Array): Boolean
  - updateIngredientList(Array): Boolean
  - deleteIngredientList(String): Boolean

Instruction Table:

- Tactic(s):
  - Adhere to standards: Follow internal standards for database table norms will help ensure ACID or any other database consistency metrics we deem important.
- Interface(s):
  - findInstruction(String): Array
  - insertInstruction(Array): Boolean
  - updateInstruction(Array): Boolean
  - deleteInstruction(String): Boolean

Device Level Authentication System:

- Tactic(s):
  - Adhere to standards: Use what the OS recommends.
  - Abstracting common services: It may be worth it to create an authorization system to manage all authentication.
- Interface(s):
  - authenticateDevice(String): Boolean
  - getDeviceAuthStatus(): Boolean
  - invalidateDeviceAuth(): Boolean

Render Stored Recipe List Action:

- Tactic(s):
    - Abstracting common services: Rendering of recipes will occur in multiple spots so it will be useful to make any rendering usable by many systems.
- Interface(s):
    - renderRecipeList(Array): Boolean

Recipe Select Action:

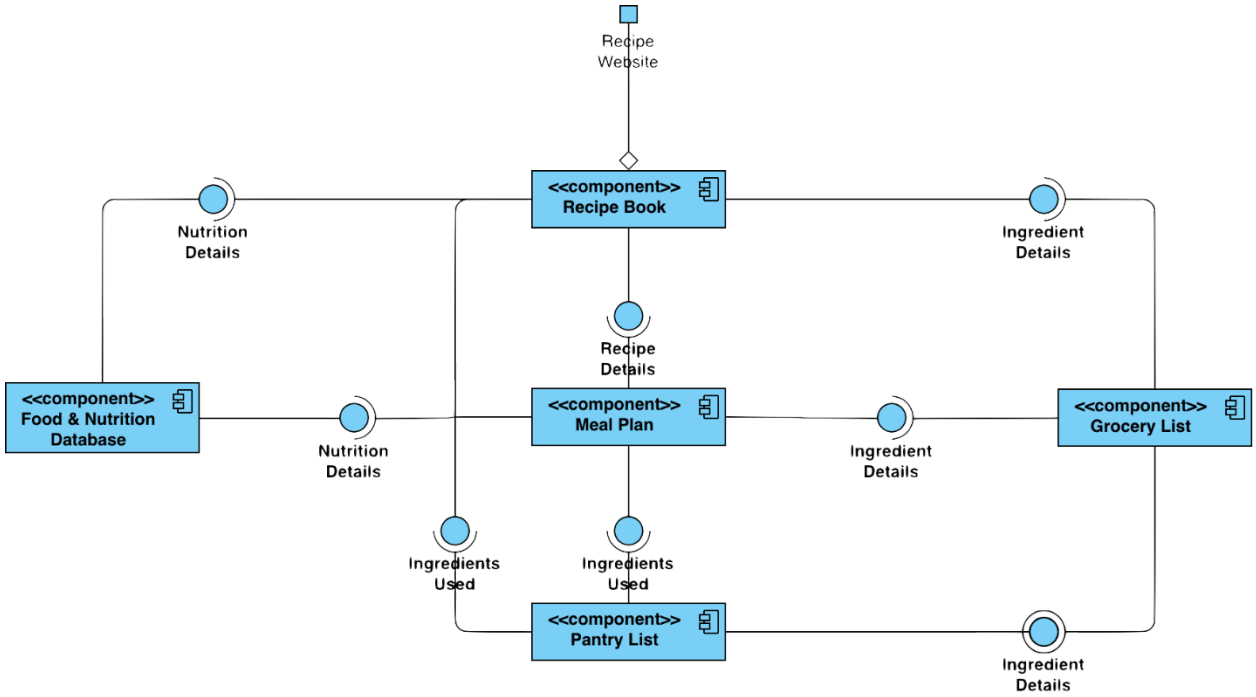
- Tactic(s):
    - Maintain User Task Model: Helping users select a recipe quickly by offering suggestions or auto-completing based on their input.
    - Abstracting Common Services: Ensures that the recipe selection functionality is reusable across the app, allowing consistency in behavior.
- Interface(s):
    - selectRecipe(String): Object

Render System Navigation Action:

- Tactic(s):
    - Abstracting Common Services: Ensures the system navigation rendering logic is centralized and reusable across various parts of the app, making it easier to maintain and update.
- Interface(s):
    - renderNavigationMenu(Array): Boolean
    - highlightActiveNavItem(String): Boolean

Views

Component Diagram



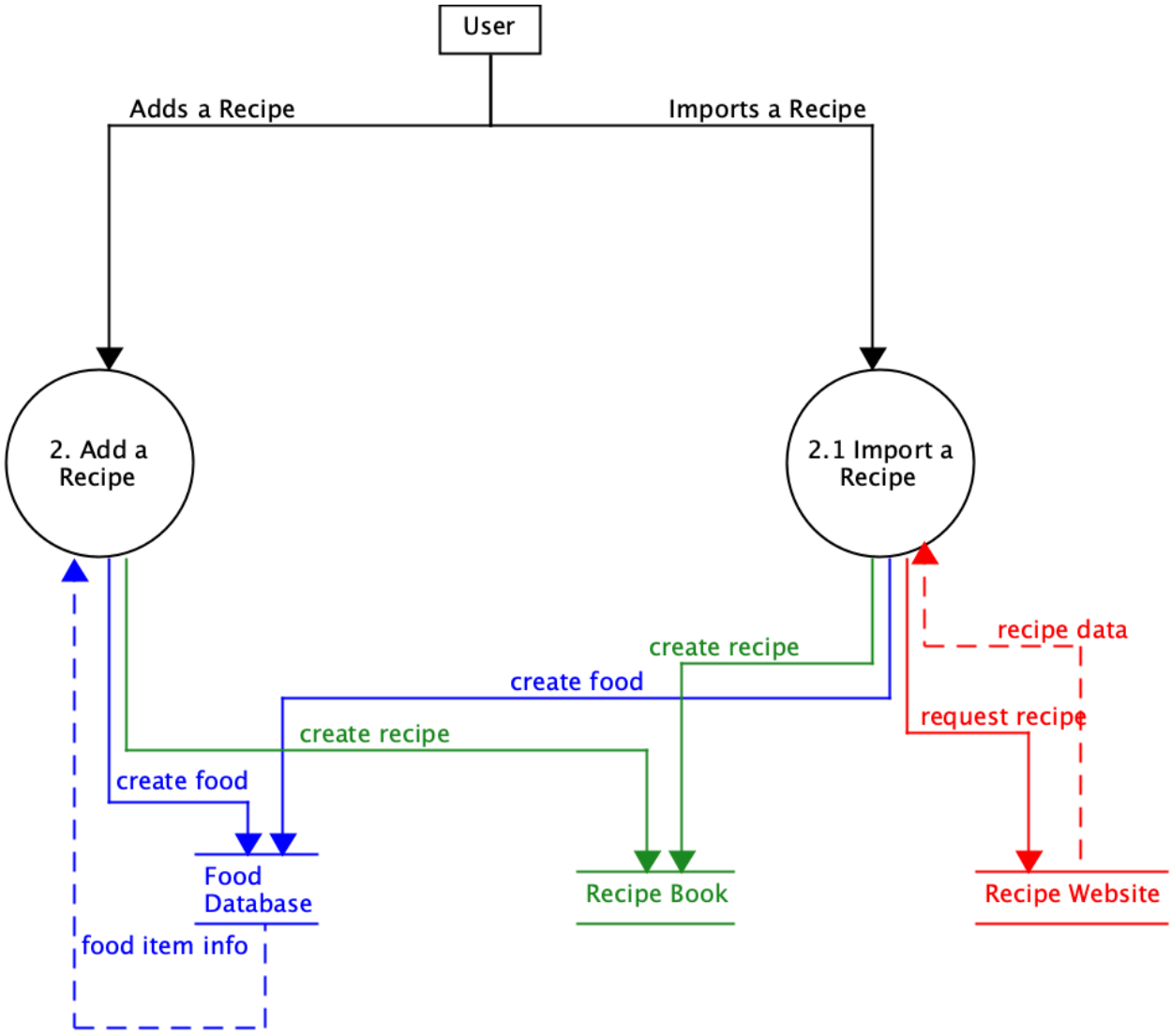
This component diagram shows the interactions between the five primary components of the system and the interfaces used in the interactions. We can identify a few key Interface models such as Nutrition, Recipe, and Ingredient which would then be composed into the correct data during runtime.

Data Dictionary

Data Element	Description	Composition or Data Type	Length	Values
Recipe	A collection of ingredients, steps, and meta-data about a recipe. Nutrition and dietary restrictions will be computed rather than stored.	RecipeName + RecipeVersion + CookTime + CookingMethod + {1:}RecipeTag + {1:}RecipeIngredient + {1:}RecipeInstruction + RecipeYield		
RecipeName	the name of the recipe	alphanumeric characters	32	
RecipeVersion	the version of the recipe	alphanumeric characters	8	
CookTime	the estimated amount of time a recipe will take to cook	alphanumeric characters	11	12D34H 56M
CookingMethod	A selector of the primary cooking method	alphabetic characters	32	Frying, Sautéing, Baking, Grilling, Stewing
RecipeTag	A descriptor of the recipe. These will be added to by the user, but there will be initialized tags for course and cuisine	alphabetic characters	64	Appetizer, Main, Side, Dessert, Breakfast, Lunch, Dinner, Snack, Pasta, Sandwich, Salad
RecipeIngredient	The foundation or branded food item and the associated quantity.	FoundationFoodItem + BrandedFoodItem + FoodQuantity		
RecipeInstruction	The text of the instruction of the recipe	alphanumeric characters	2048	
RecipeYield	The expected number of servings	numeric characters	2	
FoundationFoodItem	General food items that are not necessarily branded and can be grouped to be a single item. It can have one or more "Input Items" which would reference other Foundation Food Items, i.e. Pasta is composed of Flour and Eggs	FDCID + Description + FoodCategory + {1:}Nutrients + FoodPortion + {1:}FoundationFoodItems		
BrandedFoodItem	Items that are packaged to be bought in a grocery store	FDCID + BrandOwner + Description + GTINUPC + Ingredients + ServingSize + ServingSizeUnit + FoodCategory + {1:} FoodNutrients + LabelNutrients		
FoodQuantity	The amount of a food item	FoodSize + FoodUnit		
FDCID	Identifying number for FDC	integer	10	
Description	Usually the name of the food, but can contain other information	alphabetic characters	256	
FoodCategory	General category of food	alphabetic characters	256	Vegetables, Fruits, Canned Goods, etc
Nutrient	Micro Nutrient Data	NutrientID + NutrientNumber + NutrientName + NutrientUnit		
FoodPortion	String containing full text of portion size	alphanumeric characters	32	
NutrientID	USDA ID number for the given nutrient	integer	16	
NutrientNumber	The amount of the nutrient in the food item	numeric characters	16	
NutrientName	The name of the nutrient in the food item	alphabetic characters	64	
NutrientUnit	The unit for the number of the nutrient	alphabetic characters	8	
BrandOwner	String containing the name of the owner of the brand	alphanumeric characters	64	
GTINUPC	Bar Code Value	integer	15	
Ingredients	String containing full list of ingredients as reported by brand	alphabetic characters	2048	
ServingSize	Number for serving size	integer	3	
ServingSizeUnit	Unit for Serving Size	alphabetic characters	8	
LabelNutrients	The nutrient information found on the Nutrition Label on the Branded Food Item's packaging	Fat + SaturatedFat + TransFat + Cholesterol + Sodium + Carbohydrates + Fiber + Sugars + Protein + Calcium + Iron + Potassium + Calories		
Fat	The amount of total fat in grams	numeric characters	6	
SaturatedFat	the amount of saturated fat in grams	numeric characters	6	
TransFat	the amount of trans fat in grams	numeric characters	6	
Cholesterol	the amount of cholesterol in grams	numeric characters	6	
Sodium	the amount of sodium in grams	numeric characters	6	
Carbohydrates	the amount of carbohydrates in grams	numeric characters	6	
Fiber	the amount of fiber in grams	numeric characters	6	
Sugars	the amount of sugars in grams	numeric characters	6	
Protein	the amount of proteins in grams	numeric characters	6	
Calcium	the amount of calcium in grams	numeric characters	6	
Iron	the amount of iron in grams	numeric characters	6	
Potassium	the amount of potassium in grams	numeric characters	6	
Calories	the amount of calories in kilocalories	numeric characters	6	

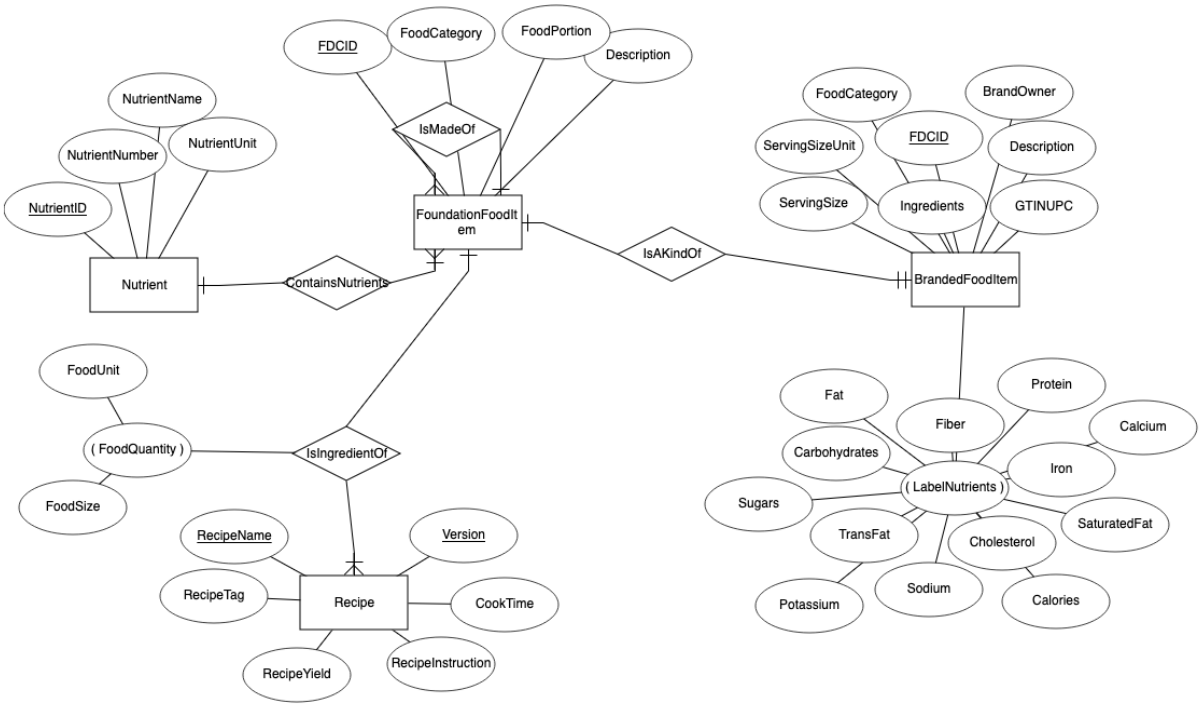
This data dictionary covers all the data required for implementing UC-2 where the user can add a new recipe to their recipe book. This diagram will enable parallel development of different logical components working with the same data structures.

Data Flow Diagram



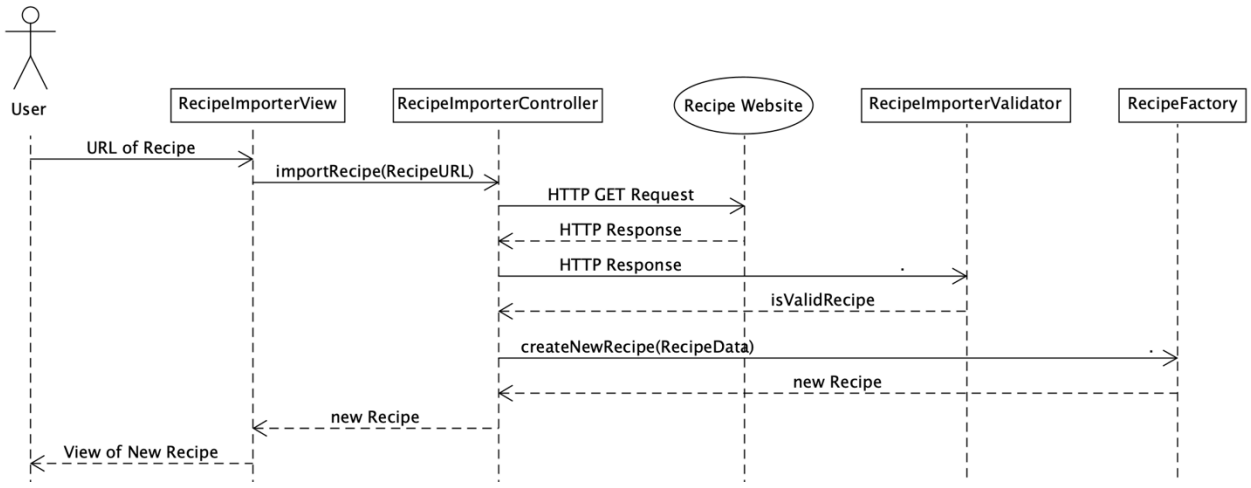
This data flow diagram shows how the data will be moving through the application during UC-2 and the different data sources required.

Entity Relationship Diagram



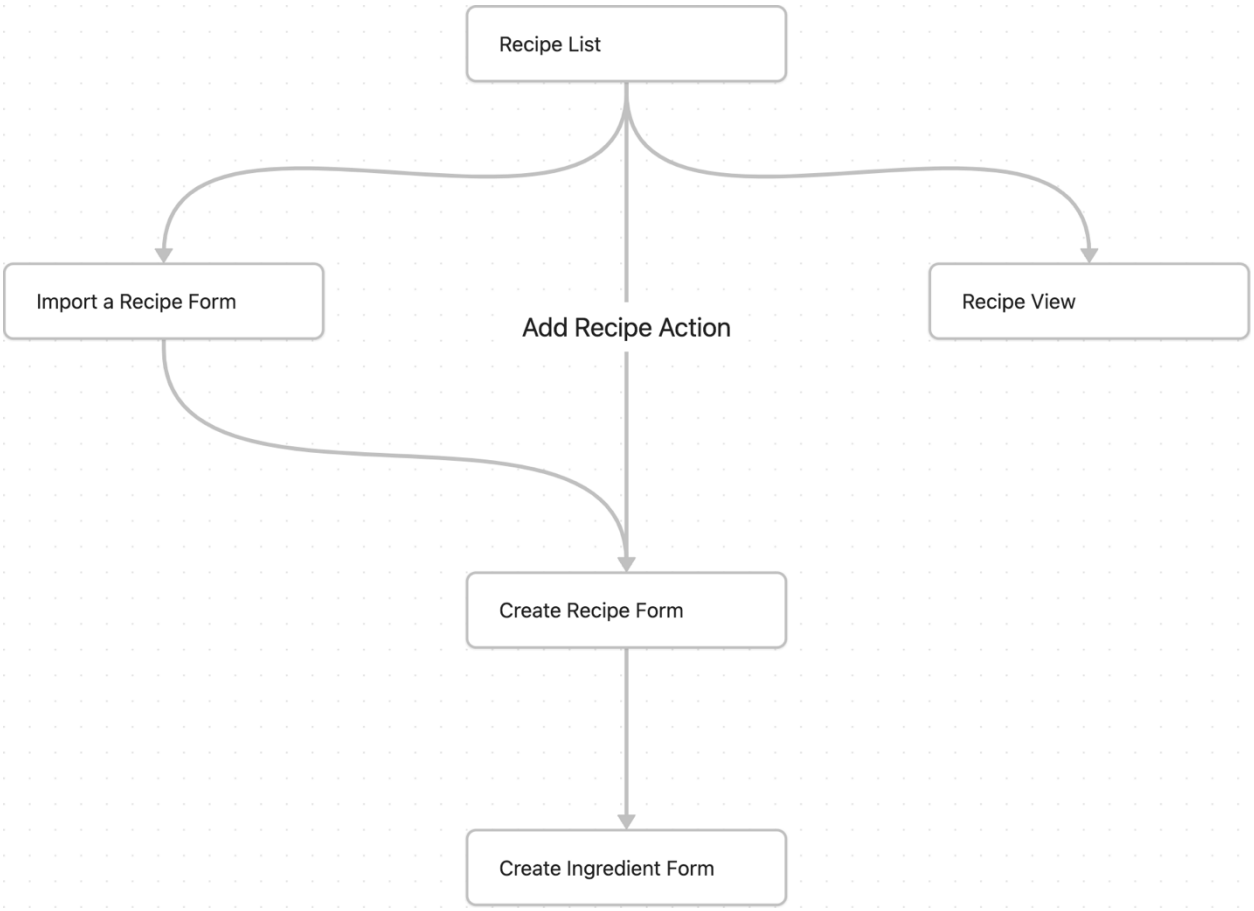
This entity-relationship diagram both enables the creation of a central data repository by defining the relationships between the data entities of the system and enables further parallel development when creating the model interfaces and shows the boundaries between them.

Sequence Diagram



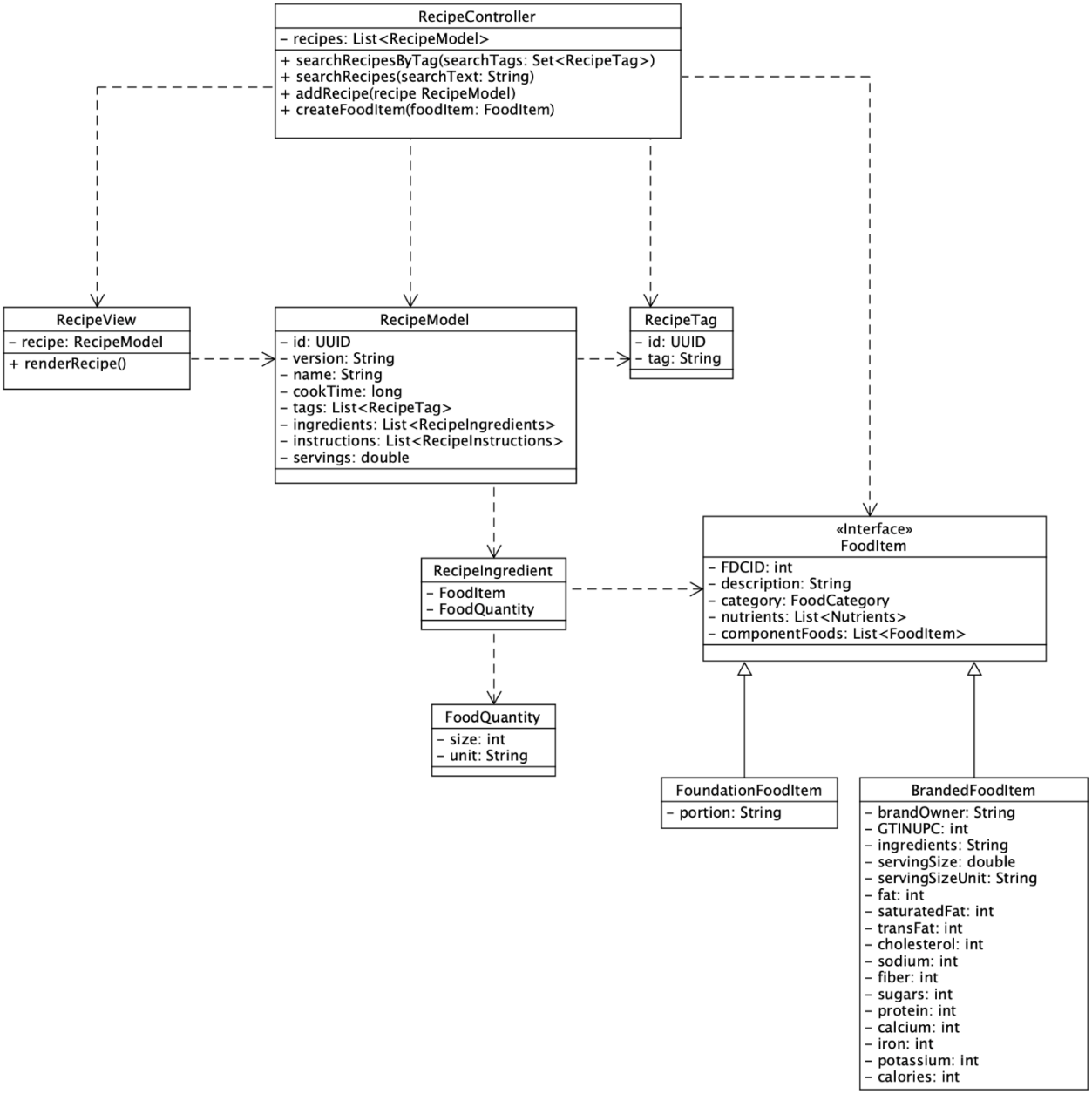
This sequence diagram shows the logical flow of the program and the calls between the modules during the import of a recipe.

Site Map



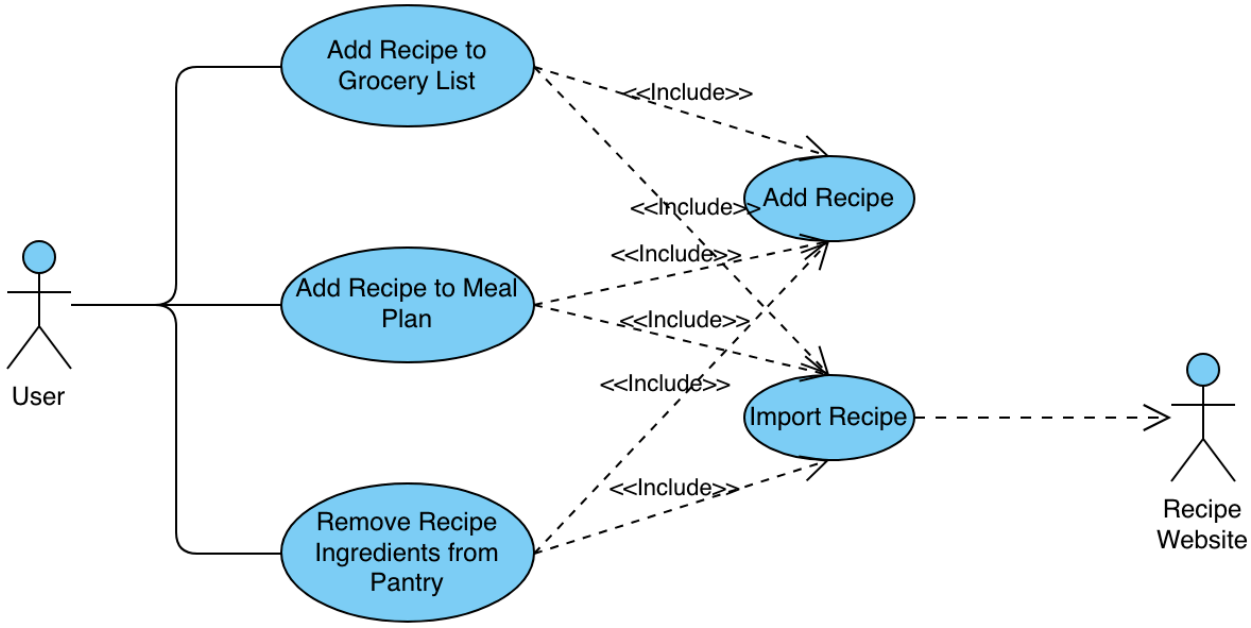
This site map shows the different UI screens and how the user can get from one to the next.

UML/Class Diagram



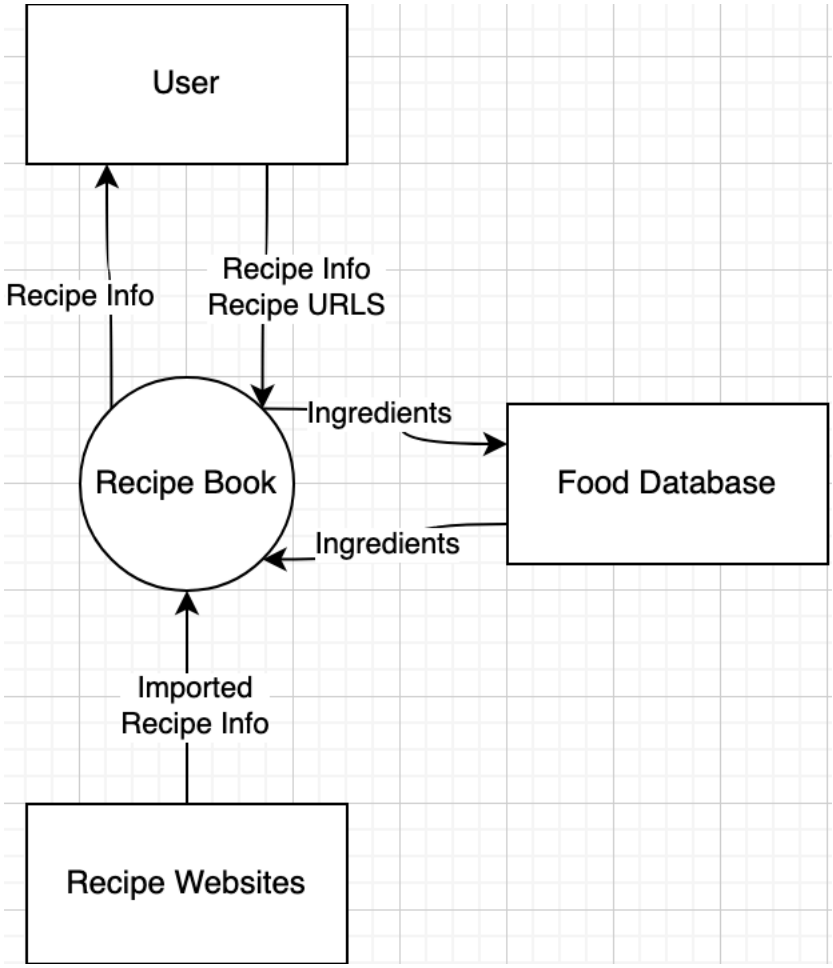
This slice of a UML Diagram shows the relationship of the Recipe Model, View, and Controller, and how it relates to other Models as a Collection of FoodItems.

Use Case Diagram – Alternate Routes to Adding/Importing Recipes



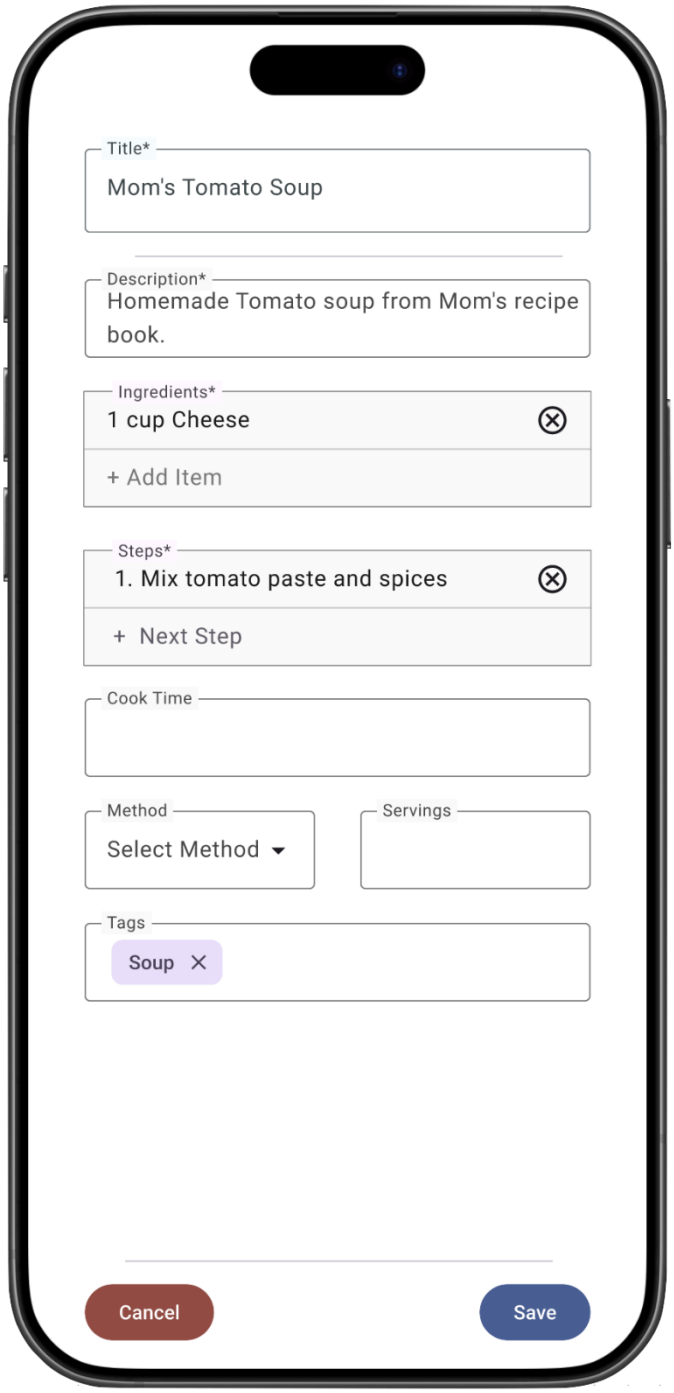
This Use Case Diagram shows the alternate routes a user can take to initiate UC-2 and UC-2.1.

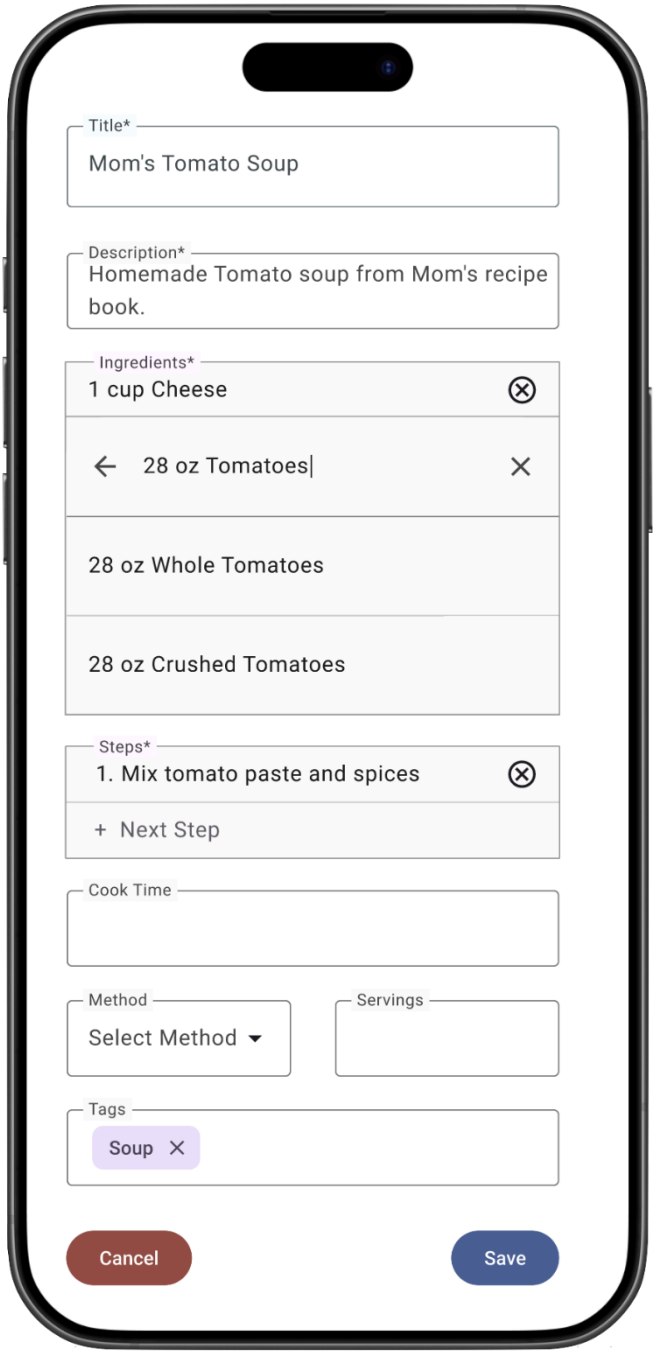
Work Context Diagram



This work context diagram shows how the Recipe Book Component interacts with external components such as the User, Recipe Websites, and The Food Database.

Add Ingredients to Recipe Mocks





Glossary

- **Device**: Supported devices for Fill My Fridge are Android or IOS phones.
- **File System**: A method and structure used by an operating system to manage and organize data on storage devices, such as hard drives or SSDs. It includes how files are named, stored, and retrieved.
- **FMF**: The app Fill My Fridge whose development specifications are defined in this document.
- **Relative Location**: A position or path described in relation to a reference point rather than an absolute coordinate. For example, specifying a file's location within a directory without referring to the full file path.
- **Recipe Database**: A database that is only accessible and written per user for storage on their personal device. This database holds all of the recipe entries that the user creates when adding a new recipe, including information such as recipe title and ingredients.
- **USDA Database**: A publicly accessible database provided by the U.S. Department of Agriculture, containing detailed nutritional information and other data about various food items. It is often used for research, meal planning, or dietary analysis.
- **XML (Extensible Markup Language)**: A flexible, text-based markup language used to store, transport, and organize data in a structured format. XML is both human-readable and machine-readable, making it useful for data exchange across systems.